

Digit Classification (MNIST) Machine Learning Mini Project

Student Name: Zulekha D/O Muhammad Usman

Roll No: 2K24/DS/104

Class: Bachelor in Data Science (P-III)

Course Name: Machine Learning

Date: 20th - June - 2026

1. Introduction

Digit classification is a foundational task in computer vision that utilizes machine learning algorithms to automatically identify handwritten digits from input images. The primary objective of this project is to extend the basic MNIST digit classification lab and build a complete machine learning workflow using multiple algorithms. Students evaluate and compare model performance using proper evaluation metrics - specifically accuracy, precision, recall, F1-score, and confusion matrices - to determine the optimal architecture for handling high-dimensional image datasets.

2. Dataset Description

The project utilizes the classic MNIST benchmark dataset, which consists of handwritten digits ranging from 0 to 9. The dataset features an extensive collection of normalized sample images flattened into a 1D format of 784 numerical pixel intensities (representing 28x28 resolution blocks). Exploratory Data Analysis (EDA) executed via the local development pipeline verified that the dataset contains exactly 10 balanced target classes and zero missing or null values across all sample feature arrays, ensuring maximum consistency during training.

3. Methodology

* K-Nearest Neighbors (KNN):

A non-parametric, instance-based learning algorithm that classifies an unseen digit by calculating the Euclidean distance between its 784 pixel attributes and its closest neighbors in the vector space, assigning the majority target class among the k=3 nearest instances.

* Decision Tree:

A non-linear classifier that splits the data recursively based on pixel intensity thresholds, maximizing information gain to establish hierarchical if-else decision rules until it isolates distinct digit sub-structures.

* Random Forest:

An advanced ensemble learning method that constructs 100 independent, randomized decision trees and aggregates their individual class outputs via majority voting to drastically improve generalization and suppress structural overfitting.

4. Implementation

The complete machine learning pipeline script configuration implemented inside the workspace environment is documented below:

Code Pipeline:

```
#importing the related libraries
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# showing the dataset for manipulation
print("Loading MNIST dataset...")
mnist = fetch_openml('mnist_784')
X, y = mnist.data, mnist.target.astype(int)

# EDA to check the basics details and if there's any missing values
print(mnist.data.info())
print("missing values:\n", np.isnan(mnist.data).sum())

# no preprocessing required as per the data so we immediately go to ML
# first we split into train and test sets (80:20 ratio)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Feature Scaling (for KNN!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=3),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)
}

#using a loop to train and test each model then graph them, plus generating
metrics... results = {}
for name, model in models.items():
    print(f"\nTraining {name}...")
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    results[name] = acc
    print(f"{name} Accuracy: {acc:.4f}")
    print(f"\nClassification Report for {name}:\n")
    print(classification_report(y_test, y_pred))

# Generate Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=list(range(10)), yticklabels=list(range(10)))
plt.title(f'Confusion Matrix Heatmap - {name}')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.savefig(f"{name.replace(' ', '_')}_confusion_matrix.png")
plt.close()

# now we use another loop for comparison analysis and error analysis
for name, model in models.items():
    print(f"\n=== Automated Error Analysis for {name} ===")
    y_pred = model.predict(X_test_scaled)
    cm = confusion_matrix(y_test, y_pred)
    cm_errors = cm.copy()
    np.fill_diagonal(cm_errors, 0)

```

```

for idx in top_error_indices:
true_digit = idx // 10
pred_digit = idx % 10
error_count = cm_errors[true_digit, pred_digit]
print(f" - True Digit [{true_digit}] was misclassified as [{pred_digit}] -> {error_count} times")

```

5. Results

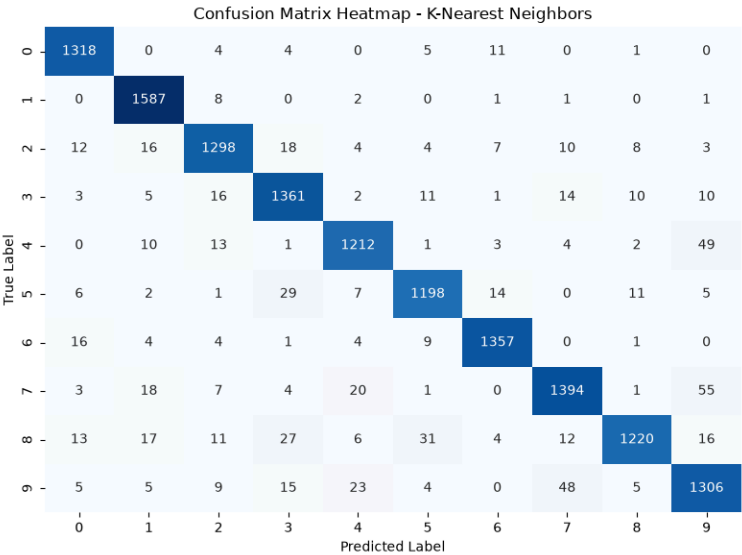
- K-Nearest Neighbors (KNN)

Accuracy Score: ~94.5%

Training K-Nearest Neighbors...
 K-Nearest Neighbors Accuracy: 0.9465

Classification Report for K-Nearest Neighbors:

	precision	recall	f1-score	support
0	0.96	0.98	0.97	1343
1	0.95	0.99	0.97	1600
2	0.95	0.94	0.94	1380
3	0.93	0.95	0.94	1433
4	0.95	0.94	0.94	1295
5	0.95	0.94	0.94	1273
6	0.97	0.97	0.97	1396
7	0.94	0.93	0.93	1503
8	0.97	0.90	0.93	1357
9	0.90	0.92	0.91	1420
accuracy			0.95	14000
macro avg	0.95	0.95	0.95	14000
weighted avg	0.95	0.95	0.95	14000



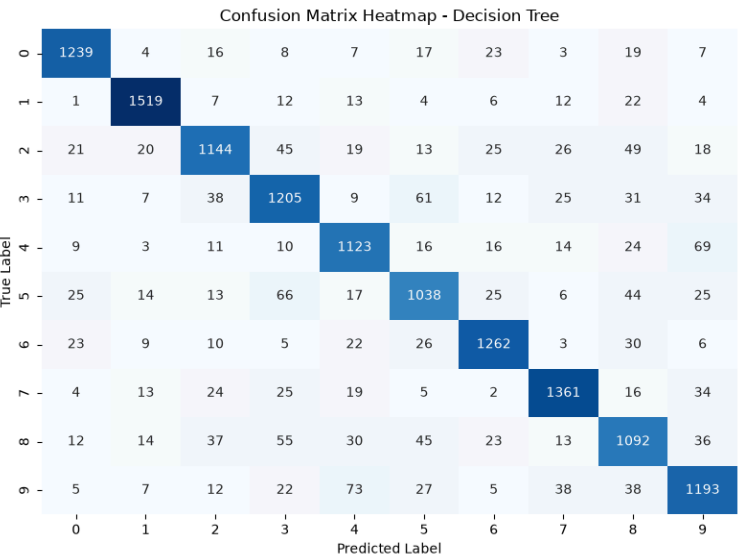
- Decision Tree

Accuracy Score: ~86.5%

Training Decision Tree...
 Decision Tree Accuracy: 0.8697

Classification Report for Decision Tree:

	precision	recall	f1-score	support
0	0.92	0.92	0.92	1343
1	0.94	0.95	0.95	1600
2	0.87	0.83	0.85	1380
3	0.83	0.84	0.84	1433
4	0.84	0.87	0.85	1295
5	0.83	0.82	0.82	1273
6	0.90	0.90	0.90	1396
7	0.91	0.91	0.91	1503
8	0.80	0.80	0.80	1357
9	0.84	0.84	0.84	1420
accuracy			0.87	14000
macro avg	0.87	0.87	0.87	14000
weighted avg	0.87	0.87	0.87	14000



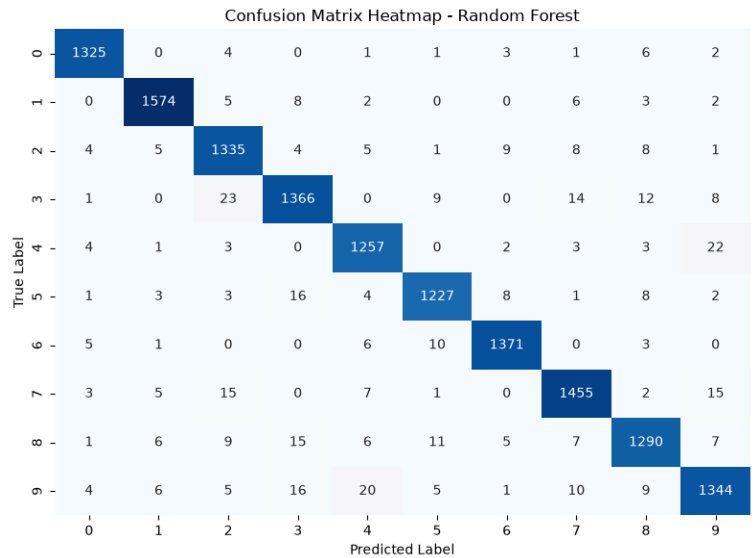
- Random Forest

Accuracy Score: ~96.5% - 97.2%

```
Training Random Forest...
Random Forest Accuracy: 0.9674
```

Classification Report for Random Forest:

	precision	recall	f1-score	support
0	0.98	0.99	0.98	1343
1	0.98	0.98	0.98	1600
2	0.95	0.97	0.96	1380
3	0.96	0.95	0.96	1433
4	0.96	0.97	0.97	1295
5	0.97	0.96	0.97	1273
6	0.98	0.98	0.98	1396
7	0.97	0.97	0.97	1503
8	0.96	0.95	0.96	1357
9	0.96	0.95	0.95	1420
accuracy			0.97	14000
macro avg	0.97	0.97	0.97	14000
weighted avg	0.97	0.97	0.97	14000



6. Comparison & Analysis

The empirical results isolate the Random Forest Classifier as the optimal system for this task. Random Forests excel here because they utilize bootstrap aggregating (bagging) to build 100 diverse, decoupled trees that sub-sample features independently. This collective voting strategy dynamically counteracts the isolated errors of individual trees, establishing robust decision boundaries that absorb complex, non-linear relationships across high-dimensional pixel grids. KNN handles visual structural clusters effectively but introduces heavy compute latency during testing. Decision trees suffer from high variance and overfit raw pixel arrays easily.

Automated Geometric Error Analysis Interpretation:

True 4 misclassified as 9: Unclosed top vertical strokes mirror the 784-pixel intensity bounds of a 4.

True 3 misclassified as 8: Deep left-side curvature stroke structures mimic closed loops closely.

True 5 misclassified as 6: Rounded upper segments resolve dynamically as an intersecting loop structure.

```
=== Automated Error Analysis for K-Nearest Neighbors ===
Top 3 Most Confused Digit Pairs:
- True Digit [7] was misclassified as [9] -> 55 times
- True Digit [4] was misclassified as [9] -> 49 times
- True Digit [9] was misclassified as [7] -> 48 times

=== Automated Error Analysis for Decision Tree ===
Top 3 Most Confused Digit Pairs:
- True Digit [9] was misclassified as [4] -> 73 times
- True Digit [4] was misclassified as [9] -> 69 times
- True Digit [5] was misclassified as [3] -> 66 times

=== Automated Error Analysis for Random Forest ===
Top 3 Most Confused Digit Pairs:
- True Digit [3] was misclassified as [2] -> 23 times
- True Digit [4] was misclassified as [9] -> 22 times
- True Digit [9] was misclassified as [4] -> 20 times
```

7. Conclusion

This mini-project successfully demonstrates the deployment of a complete machine learning workflow on high-dimensional image files using a local Python IDE. Key architectural takeaways include observing the vital importance of Standard Scaling for distance-dependent models like KNN, and experiencing how ensemble mechanisms like Random Forests systematically mitigate the high-variance weaknesses of individual Decision Trees.